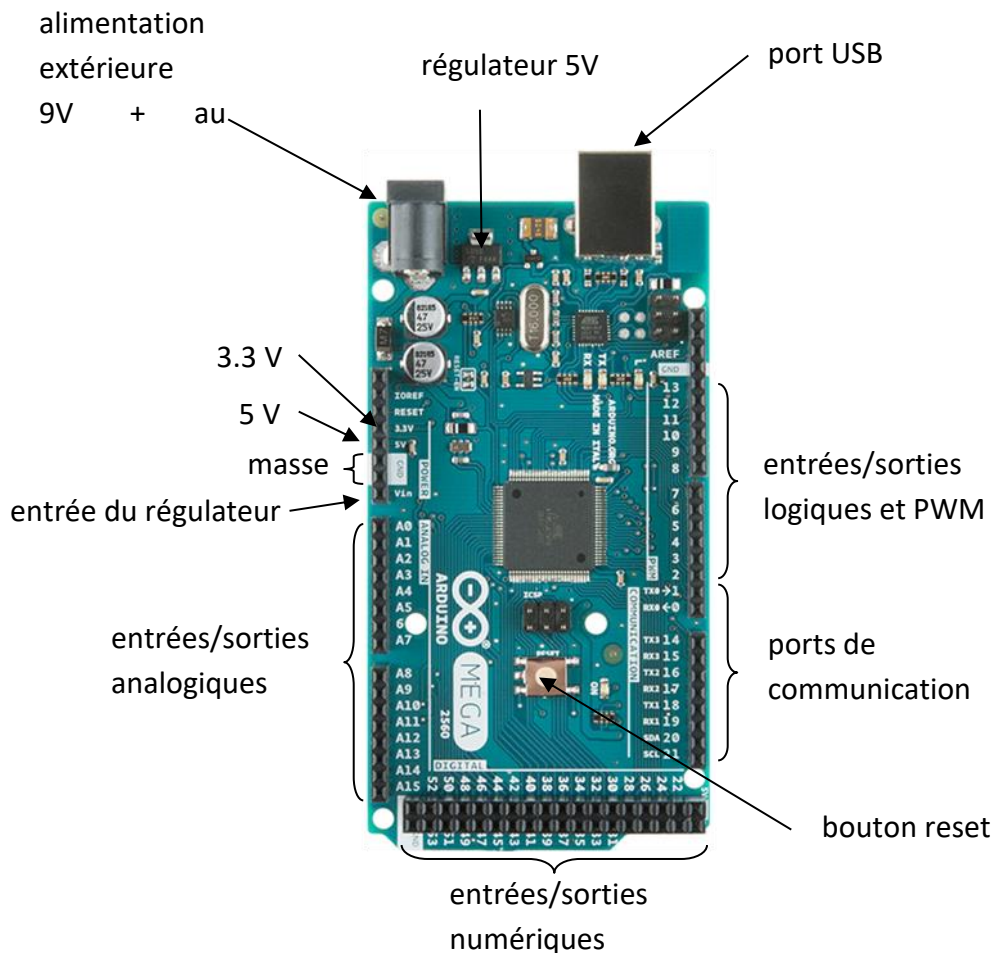


I. Prise en main de la carte ARDUINO MEGA :

I.1 Présentation de la carte ARDUINO MEGA :

La carte Arduino Mega 2560 est une carte à microcontrôleur basée sur un ATmega2560 (Atmel).



Cette carte dispose :

- de 54 broches numériques d'entrées/sorties (dont 14 peuvent être utilisées en sorties PWM (largeur d'impulsion modulée)),
- de 16 entrées analogiques (qui peuvent également être utilisées en broches entrées/sorties numériques),
- de 4 UART (port série matériel),
- d'un quartz 16Mhz,
- d'une connexion USB,
- d'un connecteur d'alimentation jack,
- d'un connecteur ICSP (programmation "in-circuit"),
- et d'un bouton de réinitialisation (reset).

Synthèse des caractéristiques

Microcontrôleur	ATmega2560
Tension de fonctionnement	5V
Tension d'alimentation (recommandée)	7-12V
Tension d'alimentation (limites)	6-20V
Broches E/S numériques	54 (dont 14 disposent d'une sortie PWM)
Broches d'entrées analogiques	16 (utilisables en broches E/S numériques)
Intensité maxi disponible par broche E/S (5V)	40 mA
Intensité maxi disponible pour la sortie 3.3V	50 mA
Intensité maxi disponible pour la sortie 5V	Fonction de l'alimentation utilisée - 500 mA max si port USB utilisé seul
Mémoire Programme Flash	256 KB dont 8 KB sont utilisés par le bootloader
Mémoire SRAM (mémoire volatile)	8 KB
Mémoire EEPROM (mémoire non volatile)	4 KB
Vitesse d'horloge	16 Hz

I.2 Structure d'un programme écrit sous Arduino :

La programmation des microcontrôleurs en utilisant l'environnement Arduino est très proche de celle de la programmation en C. Le programme est lu par le microcontrôleur de haut vers le bas. Une variable doit être déclarée avant d'être utilisée par une fonction.

La structure minimale est constituée de différentes parties :

- **entête** : déclaration des variables, des constantes, indication de l'utilisation de bibliothèques etc...
- un **setup** (= initialisation) cette partie n'est lue qu'une seule fois, elle comprend les fonctions devant être réalisées au démarrage (utilisation des broches en entrées ou en sortie, mise en marche du port série, de l'I2C etc...).
- une **loop** (boucle) : cette partie est lue en boucle ! C'est ici que les fonctions sont réalisées.

En plus de cette structure minimale, on peut ajouter :

– des « **sous-programmes** » ou « **routines** » qui peuvent être appelées à tout moment dans la boucle, très pratique pour réaliser des morceaux de codes répétitifs.

– Des « **callbacks** », ce sont des fonctions qui sont rappelées automatiquement depuis une bibliothèque.

I.3 La syntaxe du langage Arduino

Voir la page d'accueil du document références (menu "Aide" du logiciel arduino), ou directement sur le site internet, qui renvoie vers les explications de chaque commande de la syntaxe Arduino et dont voici la table des matières. Chaque instruction est suivie de sa traduction, entre-parenthèses et en noir.

Structure générale

- **void setup()** (configuration-préparation)
- **void loop()** (exécution)

Contrôle et conditions

- **if** (si...)
- **if...else** (si...alors...)
- **for** (pour...)
- **switch case** (dans le cas où...)
- **while** (pendant que ...)

Opérations de comparaison

- **==** (équivalent à)
- **!=** (différent de)
- **<** (inférieur à)
- **>** (supérieur à)
- **<=** (inférieur ou égal à)
- **>=** (supérieur ou égal à)

Operations booléennes

- **&&** (et)
- **||** (ou)

- **!** (et pas)

Autres commandes

- **//** (commentaire simple ligne)
- **/* */** (commentaire multi-lignes)
- **#define** (donner une valeur à un nom)

Variables

- **char** (variable 'caractère')
- **int** (variable 'nombre entier')
- **long** (variable 'nombre entier de très grande taille')
- **string** (variable 'chaîne de caractères')
- **array** (tableau de variables)

Niveaux logiques des connecteurs numériques

- **HIGH** (état 1)
- **LOW** (état 0)
- **INPUT** (configuré en entrée)
- **OUTPUT** (configuré en sortie)

Fonctions

Entrées-sorties numériques

- **pinMode(broche, état)** (configuration des broches)
- **digitalWrite(broche, état)** (écrire un état sur une broche num.)
- **digitalRead(broche)** (lire un état sur une broche num.)
- **unsigned long pulseIn(broche, état)** (lire une impulsion sur une broche num.)

Entrées analogiques

- **int analogRead(broche)** (lire la valeur d'une broche ana.)
- **analogWrite(broche, valeur)** (PWM : écrire une valeur analogique sur les broches 9, 10 ou 11)

Gestion du temps

- unsigned long `millis()` (temps de fonctionnement du programme)
- `delay(ms)` (attente, en millisecondes)
- `delayMicroseconds(us)` (attente, en microsecondes)

I.4 Exemple d'un programme : commander une sortie du microcontrôleur.

Un programme utilisateur Arduino est une suite d'instructions élémentaires sous forme textuelle, ligne par ligne. La carte lit puis effectue les instructions les unes après les autres, dans l'ordre défini par les lignes de code.

Structure d'un programme

Il y a trois phases consécutives:

Commentaires multilignes pour se souvenir du patch ==>

1/La définition des constantes et des variables

```
int ledPin = 13; // LED connectée à la broche 13
```

2/La configuration des entrées et sorties

`void setup()`

```
void setup()
{
  pinMode(ledPin, OUTPUT); // configure ledPin comme une sortie
}
```

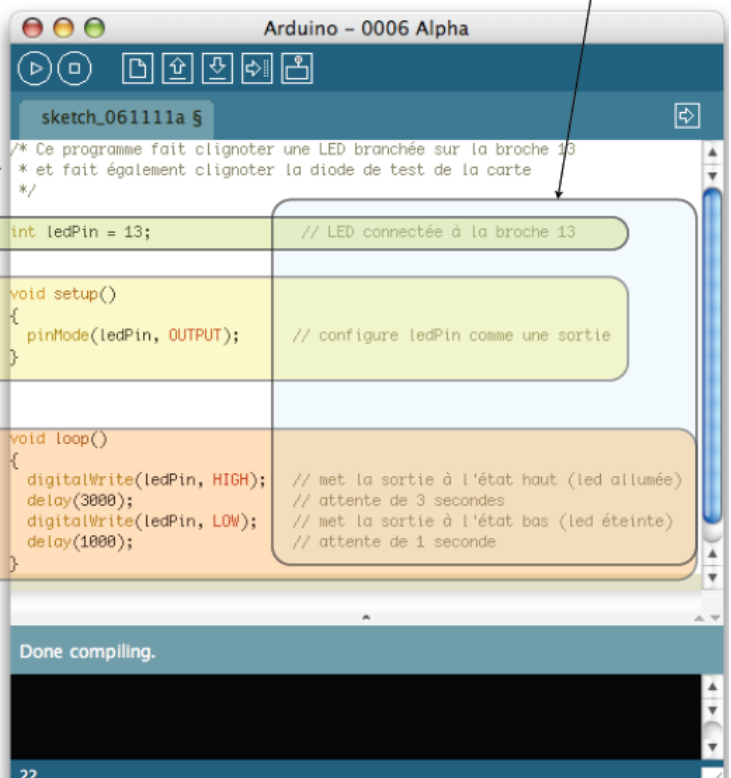
3/La programmation des interactions et comportements

`void loop()`

```
void loop()
{
  digitalWrite(ledPin, HIGH); // met la sortie à l'état haut (led allumée)
  delay(3000); // attente de 3 secondes
  digitalWrite(ledPin, LOW); // met la sortie à l'état bas (led éteinte)
  delay(1000); // attente de 1 seconde
}
```

Une fois la dernière ligne exécutée, la carte revient au début de la troisième phase et recommence sa lecture et son exécution des instructions successives. Et ainsi de suite.

Cette **boucle** se déroule des milliers de fois par seconde et anime la carte.



LES MICROCONTROLEURS

Prise en main de la carte ARDUINO

Commentaires

Toujours écrire des commentaires sur le programme: soit en multiligne, en écrivant entre des `/* */`, soit sur une ligne de code en se séparant du code avec `//`

Définition des variables:

Pour notre montage, on va utiliser une sortie numérique de la carte, qui est par exemple la 13^{ème} sortie numérique. Cette variable doit être définie et nommée ici: on lui donne un nom arbitraire `BrocheLED`. Le mot de la syntaxe est pour désigner un nombre *entier* est `int`

Configuration des entrées-sorties `void setup()`:

Les broches numériques de l'Arduino peuvent aussi bien être configurées en entrées numériques ou en sorties numériques. Ici on va configurer `BrocheLED` en sortie. `pinMode ( nom, état )` est une des quatre fonctions relatives aux entrées-sorties numériques.

Programmation des interactions `void loop()`:

Dans cette boucle, on définit les opérations à effectuer, dans l'ordre:

- `digitalWrite ( nom, état )` est une autre des quatre fonctions relatives aux entrées-sorties numériques.
- `delay(temps en millisecondes)` est la commande d'attente entre deux autres instruction
- Chaque ligne d'instruction est terminée par un point virgule
- Ne pas oublier les accolades, qui encadrent la boucle.

(Syntaxe en marron, paramètres utilisateur en vert)

```
/* Ce programme fait clignoter une LED branchée sur la broche 13
 * et fait également clignoter la diode de test de la carte
 */

int BrocheLED = 13; // Définition de la valeur 13 et du nom de la broche à
                    // utiliser

void setup()
{
  pinMode(BrocheLED, OUTPUT); // configure BrocheLED comme une
                              // sortie
}

void loop()
{
  digitalWrite(BrocheLED, HIGH); // met la sortie num. à l'état haut (led
                                // allumée)
  delay(3000); // attente de 3 secondes
  digitalWrite(BrocheLED, LOW); // met la sortie num. à l'état bas (led
                                // éteinte)
  delay(1000); // attente de 1 seconde
}
```

I.5 Transfert du programme sur la carte :

```
/* avec une vitesse de clignotement proportionnelle à l'éclairement ambiant
 * capté par une cellule photo-électrique.
 * (c) 2014, Centre de Ressources Art Sensuel.
 */

int sensuel = 0; // variable identifiant le port ser. 0 de la
                // carte
int LED = 13; // variable identifiant le port num. 13 de la
              // carte
int LDR = 0; // variable identifiant la valeur de la luminosité
              // du capteur L

void setup()
{
  pinMode(LED, OUTPUT); // configure la broche 13 comme une
                        // sortie
}

void loop()
{
  LDR = analogRead(sensuel); // lire la donnée sensuel
  digitalWrite(LED, HIGH); // allumer la LED L
  delay(1000); // attendre pendant la même durée
                // que la sensuel en millisecondes
  digitalWrite(LED, LOW); // éteindre la LED L
  delay(1000); // attendre pendant la même durée
}
```



Vérifier



S'il n'y a pas d'erreurs on verra s'afficher: "done compiling", suivi de la taille du programme.



On peut sauver le fichier sur l'ordinateur, puis appuyer sur le bouton de ré-initialisation de la carte, ci-dessous.



Ré-initialiser

Télécharger

Et enfin, télécharger le programme sur l'Arduino: attention, vous avez 5 secondes après l'appui sur le bouton de ré-initialisation pour cliquer sur le bouton "Upload" !



Et voilà ! les deux petites LEDs TX RX sur la carte clignotent pendant le chargement, puis, quelques secondes plus tard, le programme se met en route.....jusqu'à ce qu'on éteigne la carte...